



UNIVERSITY OF ENGINEERING AND TECHNOLOGY, (NEW CAMPUS)

Project Proposal

Submitted by:

Malik Abdul Wahab **2023-CS-659**

Kinza Saeed **2023-CS-671**

Submitted to:

Ma'am Mariyam Manzoor

Project Title:

Mini Linux Task Manager (HTOP Clone)

Introduction:

This project aims to develop a simplified Linux Task Manager that displays system processes, CPU usage, memory status, and allows scheduling simulations. The system will read live process information from the Linux `/proc` filesystem and provide a basic interface to monitor and manage processes. The project will also include a small module to simulate CPU scheduling algorithms for better understanding of OS concepts.

Methodology:

The system will be implemented in **modular components**, each representing an OS concept.

2.1 Process Monitoring Module

- Reads real-time data from `/proc/[pid]/stat`
- Tracks: PID, CPU usage, memory usage, state, thread count
- Dynamically allocates memory for storing process table

2.2 CPU Scheduling Module

Simulates at least three scheduling algorithms:

- **FCFS (First Come First Serve)**
- **SJF (Shortest Job First)**
- **Round Robin**

Processes for scheduling will be simulated internally using custom structures.

2.3 Multithreading

Uses **pthread** library:

- Thread 1 → Process Scanner
- Thread 2 → UI/Display Updater
- Thread 3 → Scheduling Simulator

Threads share data structures, requiring synchronization.

2.4 Process Creation Module

- Allows user to launch new programs
- Uses `fork()` and `exec()` system calls
- Child process replaces its image using `exec` family

2.5 Inter-Process Communication (IPC)

- **Shared Memory** or **Message Queues**
- Used for communication between scheduler thread and process monitor thread

2.6 Synchronization

- **Mutex locks** → Protect shared process list
- **Semaphores** → Control resource access & ordering
- Ensures safe multithreaded operations

2.7 Memory Management

- All process entries are allocated using `malloc()`
- Proper `free()` cleanup during exit
- Dynamic resizing of process table

Scope:

The scope of this project includes:

- Real-time process listing
- Dynamic CPU & memory usage display
- Implementing OS scheduling algorithms
- Multithreaded data collection
- Launching processes from inside the task manager
- Communication between modules using IPC
- Full synchronization for concurrency
- Clean and modular C code